

Algorithm for file updates in Python

Project description

At my organization, the access to restricted content is controlled with an allowed list of IP addresses. The `"allow_list.txt"` file has the IP addresses that are allowed stored on it. A separate remove list identifies IP addresses that should be removed because they no longer have access to this content. I was tasked to create an algorithm to automate updating the `"allow_list.txt"` file and remove IP addresses that should no longer have access.

Open the file that contains the allow list

At the beginning of the algorithm, I opened the `"allow_list.txt"` file. I assigned this file name as a string to the `import_file` variable:

```
# Assign 'import_file' to the name of the file  
import_file = "allow_list.txt"
```

After, I had to use the `with` statement to open file:

```
# Build 'with' statement to read in the initial contents of the file  
with open(import_file, "r") as file:
```

In my algorithm, I use a `with` statement together with the `open()` function in read mode to access the allow-list file. Opening this file lets me read the IP addresses stored inside it. The `with` keyword automatically handles resource management, meaning the file will close on its own once the block of code finishes running. In the line `with open(import_file, "r") as file:`, the `open()` function takes two arguments: the first is the name of the file I want to load, and the second tells Python how I want to use it—here, `"r"` specifies that I'm opening it for reading. The `as` keyword assigns the opened file to a variable named `file`, which I can then use to work with the file's contents within the `with` block.

Read the file contents

In order to read the file contents, I used the `.read()` method to convert it into the string.

```
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()
```

When I open a file using the `open()` function the `"r"` argument, which stands for `"read"`, I'm able to use the `.read()` method inside the `with` block. The `.read()` method takes the entire file and returns its contents as a single string. I used this method on the file object created in the `with` statement and stored the resulting string in a variable called `ip_addresses`.

Overall, this section of code loads everything from the `"allow_list.txt"` file into a string so that I can later process, organize, or extract specific information from it in my python program.

Convert the string into a list

For me to be able to remove individual IP addresses from the allow list, I need to convert it to list format. I used the `.split()` method to convert the `ip_addresses` string into a list:

```
# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()
```

The `.split()` method is used by attaching it to a string variable, and its job is to break that string into a list. I'm splitting the `ip_addresses` string so that each ip address becomes its own list item, making it easier to remove or manage individual addresses in the allowed list. Since `.split()` uses whitespace as the default separator, it automatically divides the string wherever spaces appear. In this algorithm, the method takes the string stored in `ip_addresses`, which contains multiple IPs separated by spaces, and turns it into a list of separated ip address entries. I then assigned this new list back to the `ip_addresses` variable for later use.

Iterate through the remove list

The main part of my algorithm involves iterating through the IP addresses that are elements in the `remove_list`. To achieve this, I incorporated a `for` loop.

```
# Build iterative statement  
# Name loop variable 'element'  
# Loop through 'ip_addresses'  
  
for element in ip_addresses:
```

A `for` loop in Python is used to run a block of code repeatedly for each item in a sequence. In an algorithm like this, its purpose is to perform the same set of actions on every value in the list. The loop begins with the `for` keyword, followed by a loop variable—here called `element`—and then the keyword `in`. The `in` keyword tells Python to go through each item in the `ip_addresses` list one by one, assigning each IP address to the loop variable `element` as the loop runs.

Remove IP addresses that are on the remove list

In my algorithm, I needed to delete any IP address from the `ip_addresses` allow list if it also appeared in the `remove_list`. Since the `ip_addresses` list contained no duplicates, I could accomplish this task using the following code:

```
for element in ip_addresses:  
  
    # Build conditional statement  
    # If current element is in 'remove_list',  
  
    if element in remove_list:  
  
        # then current element should be removed from 'ip_addresses'  
  
        ip_addresses.remove(element)
```

Inside my `for` loop, I added a conditional check to see whether the current loop variable, `element`, existed in the `ip_addresses` list. This prevents errors that would occur if `.remove()` were called on an item that isn't in the list.

If the element was found, I then used `.remove()` on `ip_addresses`, passing in the element so that every IP address included in `remove_list` would be successfully removed from `ip_addresses`.

Update the file with the revised list of IP addresses

For the final step in my algorithm, I needed to update the allow list file with a revised list of IP addresses. To accomplish that I needed to convert the list back into a string. I used the `.join()` method for this:

```
# Convert `ip_addresses` back to a string so that it can be written into the text file  
ip_addresses = " ".join(ip_addresses)
```

The `.join()` method is used to take all the items in an iterable and combine them into a single string. It's called on a string that serves as the separator, which determines what will appear between each element when they are joined together. In this algorithm, I used `.join()` to turn the `ip_addresses` list into a formatted string so it could be passed to the `.write()` method for saving to the file `"allow_list.txt"`. I chose `" "` as the separator so that each IP address would be written on its own line.

Then, I used another `with` statement and the `.write()` method to update the file:

```
# Build `with` statement to rewrite the original file  
with open(import_file, "w") as file:  
    # Rewrite the file, replacing its contents with `ip_addresses`  
    file.write(ip_addresses)
```

In this step, I used the `"w"` argument with the `open()` function inside my `with` statement. This argument tells Python to open the file in *write mode*, meaning its existing contents will be overwritten. Using `"w"` allows me to call the `.write()` method within the `with` block. The `.write()` method takes a string and writes it to the file, replacing anything that was previously stored there.

Here, my goal was to save the updated allow list to `"allow_list.txt"`, ensuring that any IP addresses removed from the list would no longer have access. To update the file, I called `.write()` on the `file` object defined in the `with` statement and passed in the `ip_addresses` variable. This instructs Python to replace the contents of the file with the string version of the updated allow list.

Summary

I developed an algorithm to remove specific IP addresses listed in the `remove_list` variable from the approved IP address file, `"allow_list.txt"`. The process began by opening the file, reading its contents as a string, and converting that string into a list stored in `ip_addresses`. Next, I looped through each IP address in `remove_list`, checking whether it existed in `ip_addresses`. Whenever a match was found, I used the `.remove()` method to delete it from the list. After updating the list, I applied `.join()` to convert `ip_addresses` back into a single string. Finally, I wrote this updated string back into `"allow_list.txt"`, replacing the old content with the revised allow list.